Parallel & Distributed Computing

20-Minute Pre-Exam Master Notes (Lectures 1–5)

> **EXAM QUICK-START:** Focus heavily on **Amdahl's Law calculations** (Lect 1), **CPU Architectures & Speculation** (Lect 2), **Process vs. Thread & Message Passing Matrix** (Lect 3), **Logical vs. Physical Cache & DSM Approaches** (Lect 4), and **MPI Functions, Status & Deadlock Solutions** (Lect 5).

Lecture 1: Foundations, Parallel vs. Distributed & Amdahl's Law

1. Serial vs. Parallel Computing: Serial computing executes one instruction at a time sequentially on a single CPU. Parallel computing splits a problem into smaller sub-tasks (divide-and-conquer) executed simultaneously across multiple processing units.

2. Parallel vs. Distributed Systems:
• *Parallel Computing:* Uses a single system with multiple processors/cores sharing a common memory space or fast bus interconnect.
• *Distributed Computing:* Consists of multiple independent computer nodes connected via a network, each with its own private address space, communicating solely by explicit message passing.

Dimension	Parallel Computing	Distributed Computing
System Architecture	Single system, multiple cores/CPUs	Multiple independent networked nodes
Memory Model	Shared memory (or high-speed bus)	Distributed memory (private address space)
Communication	Hardware bus / Shared RAM access	Network message passing (higher latency)
Scalability & Fault Tolerance	Limited by single system bus; low fault tolerance	Highly scalable (scale-out); high fault tolerance

3. Amdahl's Law & Speedup Formulas:
• **Parallel Speedup:** $S = T_{serial} / T_{parallel}$
• **Amdahl's Law Formula:** $S = 1 / ((1 - P) + (P / N))$, where P = parallelizable fraction, $(1 - P)$ = inherently serial fraction, and N = number of processors.
• **Theoretical Maximum Speedup:** As $N \to \infty$, $S_{max} = 1 / (1 - P)$. The serial portion creates an unyielding upper bottleneck.

4. Scalability Dimensions & Scaling Models:
• *Horizontal Scaling (Scale Out):* Adding more physical computers/nodes to a network.
• *Vertical Scaling (Scale Up):* Adding more CPU, RAM, or storage inside a single existing server.
• *5 Dimensions of Scalability:* **Physical/Load** (adding resources), **Functional** (adding features without disruption), **Geographic** (expansion across WAN), **Generation** (adopting newer hardware generations), and **Heterogeneous** (multi-vendor integration).

Lecture 2: Hardware Architectures & Instruction-Level Parallelism

1. Von Neumann Architecture: Features the stored-program concept (instructions and data reside in the same physical memory) and sequential instruction execution. The single shared bus creates the classic *Von Neumann Bottleneck*.

Architecture	Parallelism Level	Hardware Mechanism	Slide Example
Scalar	1 instruction / cycle	Executes 1 instruction on 1 data element sequentially	Intel 8086
Vector	1 instr = array of ops	Operates on vector registers containing data arrays	Cray-1, Modern GPUs
Pipelined	Overlapped stages	Divides execution into stages (IF, ID, EX, WB)	MIPS R2000, ARM Cortex-M
Superscalar	Multiple instrs / cycle	Replicates functional units (ALUs) for dynamic multiple issue	Intel Pentium, ARM Cortex-A
VLIW	Multiple ops / word	Compiler packs multiple operations into one long instruction word	Intel Itanium, DSPs

2. Speculation & Branch Prediction: In multiple-issue systems, the CPU or compiler guesses the outcome of conditional branch instructions (e.g., *if (z > 0) w = x; else w = y;*) and pre-executes instructions based on the guess. If the speculation is incorrect, the processor discards the speculative results, rolls back state, and recalculates the correct path.

3. Flynn's Taxonomy (1966): Classifies computer architectures based on two information streams:
• **SISD:** Single Instruction, Single Data (Standard uniprocessor, e.g., 8086).
• **SIMD:** Single Instruction, Multiple Data (Vector processors, GPUs, array processing).
• **MISD:** Multiple Instruction, Single Data (Hypothetical / fault-tolerant pipelines).
• **MIMD:** Multiple Instruction, Multiple Data (Modern multi-core CPUs and distributed clusters).

Lecture 3: Software Architecture & Parallel Programming Models

1. Parallel Programming Models (SPMD vs. MPMD):
• **SPMD (Single Program Multiple Data):** A single executable binary runs on all processors. Each processor executes the same code, branching based on its unique rank/process ID. Most modern MPI applications use SPMD on MIMD hardware.
• **MPMD (Multiple Program Multiple Data):** Different processors execute completely different program binaries on different datasets.

Property	Process	Thread
Definition	Program in execution (heavyweight entity)	Segment/execution unit within a process (lightweight)
Address Space	Private, isolated address space	Shares parent process address space
Creation Cost	High (OS allocates dedicated memory/tables)	Low (fast creation, switching, and termination)
Communication	Explicit Message Passing (IPC / MPI)	Direct Shared Memory access

2. Dynamic vs. Static Threads:
• *Dynamic Threads:* Master thread forks worker threads on demand when work arrives; threads terminate when completed. Saves memory resources but incurs high thread creation/destruction overhead.
• *Static Threads:* A fixed thread pool is created at program startup and stays alive until shutdown. Maximizes execution performance by eliminating runtime creation overhead.

Protocol Type	Sender Blocks?	Receiver Blocks?	Buffer Exists?	Synchronization / Notes
Blocking Buffered	Only if buffer full	If buffer empty	Yes	Buffer acts as mailbox; no rendezvous required
Blocking Non-Buffered	Yes (until recv ready)	Yes (until send ready)	No	**Rendezvous Communication:** Both processes wait for handshake
Non-Blocking Buffered	No (returns immediately)	If buffer empty	Yes	Sender never waits; risk of error if buffer space insufficient
Non-Blocking Non-Buffered	No	No	No	Unreliable: Messages lost if receiver isn't ready when message arrives

Lecture 4: Memory Hierarchy & Distributed Shared Memory (DSM)

1. Virtual Memory & Cache Placement: The Memory Management Unit (MMU) translates instruction *Virtual Addresses* into *Physical Addresses* in RAM.
• **Logical (Virtual) Cache:** Placed between CPU and MMU. Uses virtual addresses. Faster access (no translation latency needed), but **must be flushed on every context switch** to prevent cross-process data leakage.
• **Physical Cache:** Placed between MMU and Main Memory. Uses physical addresses. No context-switch cache flushes required.

2. Distributed Shared Memory (DSM): An architecture where physically separated, distributed memories are addressed as a single, logically shared virtual address space.

Implementation	Level / Approach	Key Mechanism / Characteristics
Hardware-Based	On-Chip Memory	CPU-direct memory connection; extremely fast but expensive
Hardware-Based	Bus-Based Multiprocessor	Shared bus connects local caches to main memory; needs prevention algorithms
Hardware-Based	Ring-Based Multiprocessor	Token-passing ring interconnect; memory split into private and shared regions
Software-Based	Page-Based (OS-Level)	Integrated into OS virtual memory; intercepts access and maps virtual pages across nodes
Software-Based	Shared Variable / Object	External developer libraries manage explicit variable sync or object-oriented encapsulation

3. DSM Trade-offs:
• *Advantages:* Unlimited scalability (bypasses physical bus limits); hides message passing complexity; passes

complex structures by reference; leverages *locality of reference* via page transfers; cheaper than massive centralized supercomputers.
• *Disadvantages:* Slower access times than non-distributed RAM; requires complex consistency models; potential performance penalty due to page thrashing; lacks fine-grained programmer control over underlying network messages.

Lecture 5: Distributed Memory Programming with MPI

1. MPI Basics & Compilation: MPI is the standard message-passing library specification for portable distributed-memory programming.
• *Header:* #include <mpi.h>
• *Compilation:* mpicc -g -Wall -o mpi_hello mpi_hello.c
• *Execution:* mpiexec -n 4 ./mpi_hello
• *Default Communicator:* MPI_COMM_WORLD (contains all launched processes).

Function Routine	Signature & Parameters	Purpose / Semantics
MPI_Init	int MPI_Init(int *argc, char ***argv)	Initializes the MPI execution environment
MPI_Finalize	int MPI_Finalize()	Cleans up and terminates the MPI environment
MPI_Comm_size	int MPI_Comm_size(MPI_Comm comm, int *size)	Queries total process count in communicator
MPI_Comm_rank	int MPI_Comm_rank(MPI_Comm comm, int *rank)	Queries rank (0 to size-1) of calling process
MPI_Send	int MPI_Send(buf, count, type, dest, tag, comm)	Sends message buffer to destination process
MPI_Recv	int MPI_Recv(buf, count, type, src, tag, comm, status)	Receives message from source process (blocking)

2. Message Receives & Status Info:
• Receiver wildcards: MPI_ANY_SOURCE and MPI_ANY_TAG.
• *Status Structure:* status.MPI_SOURCE, status.MPI_TAG, and status.MPI_ERROR.
• *Query Received Count:* MPI_Get_count(&status, MPI_INT, &count).

3. Circular Deadlocks & Solutions:
• *Cause:* Circular wait condition where every process attempts to MPI_Send to its neighbor simultaneously before calling MPI_Recv.
• **Solution 1 (Odd/Even Partitioning):** Even processes send then receive; odd processes receive then send.
• **Solution 2 (Simultaneous Operations):** Use MPI_Sendrecv or MPI_Sendrecv_replace (uses a single buffer for send and receive).

Complete Minimal MPI Template (C):

```c
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int my_rank, comm_sz;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    printf("Process %d of %d in MPI_COMM_WORLD\n", my_rank, comm_sz);

    MPI_Finalize();
    return 0;
}
```